
UNIVERSIDAD PRIVADA DOMINGO SAVIO



LICENCIATURA EN INGENIERÍA EN SISTEMAS
"Documentacion sobre el avance en clases "

“PROGRAMACION III”

NOMBRE DEL ESTUDIANTE

Douglas Flor Hernández

DOCENTE

Gustavo Tantani Mamani

¡Creando a Firulais! Nuestra Primera Clase

C++

CODIGO I

```
#include <iostream>
#include <string>
// Definición de la CLASE 'Perro'
class Perro {
public: // Especificador de acceso (lo veremos más adelante)
    // Atributos (Variables Miembro)
    std::string nombre;
    std::string raza;
    int edad;
    // Métodos (Funciones Miembro)
    void ladrar() {
        std::cout << nombre << " dice: ¡Guau! ¡Guau!" << std::endl;
    }
    void presentar() {
        std::cout << "Hola, soy " << nombre << ", un " << raza
            << " de " << edad << " años." << std::endl;
    }
}; // ¡OJO con el punto y coma al final de la clase!
int main() {
    // Creación de OBJETOS (Instancias de la clase Perro)
    Perro perro1; // Objeto llamado perro1 de tipo Perro
    Perro perro2; // Objeto llamado perro2 de tipo Perro
    perro1.nombre = "Firulais"; // Asignando valores a los atributos de perro1
    perro1.raza = "Mestizo Jugueton";
    perro1.edad = 3;
    perro2.nombre = "Rex"; // Asignando valores a los atributos de perro2
    perro2.raza = "Pastor Aleman";
    perro2.edad = 5;

    std::cout << "--- Conozcamos a nuestras mascotas ---" << std::endl;
    perro1.presentar(); // Llamando a los métodos de los objetos
    perro1.ladrar();
    std::cout << std::endl;
    perro2.presentar();
    perro2.ladrar();
    std::cout << std::endl;
    std::cout << "--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---" << std::endl;
    return 0;
}
```

VISTO EN CONSOLA

```
input
--- Conozcamos a nuestras mascotas ---
Hola, soy Firulais, un Mestizo Jugueton de 3 años.
Firulais dice: ¡Guau! ¡Guau!

Hola, soy Rex, un Pastor Aleman de 5 años.
Rex dice: ¡Guau! ¡Guau!

--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---

...Program finished with exit code 0
Press ENTER to exit console.
```

INTRODUCIENDO MAS METODOS Y ATRIBUTOS A FIRULAIS

CODIGO II

```
#include <iostream>
#include <string>

class Perro {
public:
    // Atributos
    std::string nombre;
    std::string raza;
    int edad;
    std::string comida_favorita; CREADO
    float peso; // en kilogramos CREADO

    // Métodos
    void ladrar() {
        std::cout << nombre << " dice: ¡Guau! ¡Guau!" << std::endl;
    }

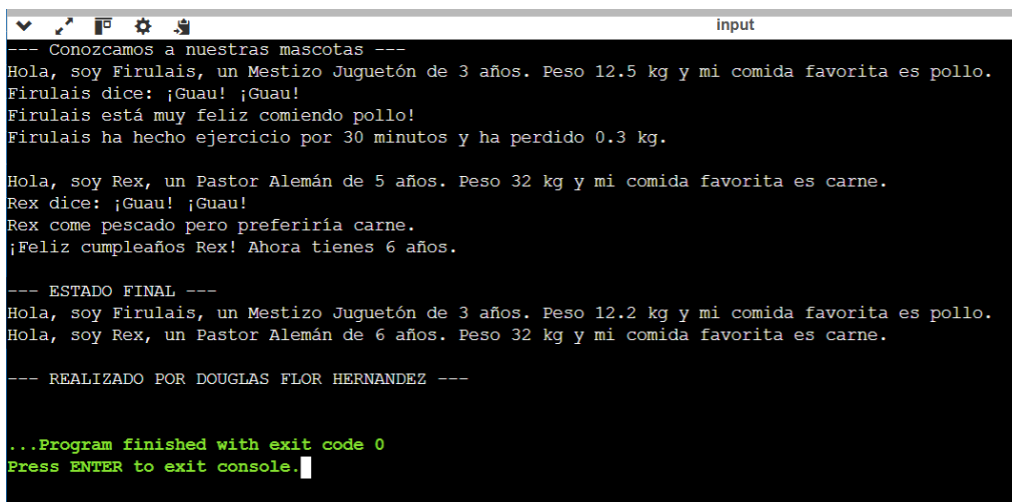
    void presentar() { MODIFICADO PARA COMIDA Y PESO
        std::cout << "Hola, soy " << nombre << ", un " << raza
            << " de " << edad << " años. Peso " << peso
            << " kg y mi comida favorita es " << comida_favorita << "." << std::endl;
    }

    void alimentar(std::string comida) {
        if(comida == comida_favorita) {
            std::cout << nombre << " está muy feliz comiendo " << comida << "!" << std::endl;
        } else {
            std::cout << nombre << " come " << comida << " pero preferiría " << comida_favorita << "." << std::endl;
        }
    }

    void hacer_ejercicio(int minutos) {
        float peso_perdido = minutos * 0.01f; // pierde aproximadamente 10g por minuto
        peso -= peso_perdido;
        std::cout << nombre << " ha hecho ejercicio por " << minutos << " minutos "
            << "y ha perdido " << peso_perdido << " kg." << std::endl;
    }
}
```

```
}  
  
void cumpleaños() {  
    edad++;  
    std::cout << "¡Feliz cumpleaños " << nombre << "! Ahora tienes " << edad << " años." << std::endl;  
}  
};  
  
int main() {  
    // Creación de objetos  
    Perro perro1;  
    Perro perro2;  
  
    // Configuración de perro1  
    perro1.nombre = "Firulais";  
    perro1.raza = "Mestizo Juguetón";  
    perro1.edad = 3;  
    perro1.comida_favorita = "pollo";  
    perro1.peso = 12.5f;  
  
    // Configuración de perro2  
    perro2.nombre = "Rex";  
    perro2.raza = "Pastor Alemán";  
    perro2.edad = 5;  
    perro2.comida_favorita = "carne";  
    perro2.peso = 32.0f;  
  
    std::cout << "--- Conozcamos a nuestras mascotas ---" << std::endl;  
    perro1.presentar();  
    perro1.ladrazar();  
    perro1.alimentar("pollo");  
    perro1.hacer_ejercicio(30);  
    std::cout << std::endl;  
  
    perro2.presentar();  
    perro2.ladrazar();  
    perro2.alimentar("pescado");  
    perro2.cumpleaños();  
    std::cout << std::endl;  
  
    std::cout << "--- ESTADO FINAL ---" << std::endl;  
    perro1.presentar();  
    perro2.presentar();  
  
    std::cout << "\n--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---" << std::endl;  
    return 0;  
}
```

VISTO EN CONSOLA



```
input  
--- Conozcamos a nuestras mascotas ---  
Hola, soy Firulais, un Mestizo Juguetón de 3 años. Peso 12.5 kg y mi comida favorita es pollo.  
Firulais dice: ¡Guau! ¡Guau!  
Firulais está muy feliz comiendo pollo!  
Firulais ha hecho ejercicio por 30 minutos y ha perdido 0.3 kg.  
  
Hola, soy Rex, un Pastor Alemán de 5 años. Peso 32 kg y mi comida favorita es carne.  
Rex dice: ¡Guau! ¡Guau!  
Rex come pescado pero preferiría carne.  
¡Feliz cumpleaños Rex! Ahora tienes 6 años.  
  
--- ESTADO FINAL ---  
Hola, soy Firulais, un Mestizo Juguetón de 3 años. Peso 12.2 kg y mi comida favorita es pollo.  
Hola, soy Rex, un Pastor Alemán de 6 años. Peso 32 kg y mi comida favorita es carne.  
  
--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Mejoras realizadas:

Nuevos atributos añadidos:

comida_favorita (string
peso (float, en kilogramos)

Nuevos métodos implementados:

alimentar(comida): Reacciona diferente según si es su comida favorita o no

hacer_ejercicio(minutos): Reduce el peso del perro según el tiempo de ejercicio

cumpleaños(): Incrementa la edad del perro y muestra un mensaje

Presentación mejorada:

El método presentar() ahora muestra también la comida favorita y el peso

Encapsulando al Estudiante

CODIGO 1

```
#include <iostream>
#include <string>
class Estudiante {
private: // DATOS PROTEGIDOS
    std::string nombre;
    int edad;
    std::string matricula;
    double promedio;
public:
    // Constructor
    Estudiante(std::string nom, int ed, std::string matr) {
        nombre = nom;
        // Usamos el setter para edad para aprovechar la validación
        setEdad(ed); // ¡Llamando a nuestro propio setter!
        matricula = matr;
        promedio = 0.0; // Promedio inicial
        std::cout << "Estudiante " << nombre << " creado." << std::endl;
    }
    // Getters
    std::string getNombre() const { return nombre; }
    int getEdad() const { return edad; }
    std::string getMatricula() const { return matricula; }
    double getPromedio() const { return promedio; }
    // Setters (con validación donde aplique)
    void setNombre(const std::string& nuevoNombre) {
        if (!nuevoNombre.empty()) {
            nombre = nuevoNombre;
        } else {
            std::cout << "Error: El nombre no puede estar vacio." << std::endl;
        }
    }
    void setEdad(int nuevaEdad) {
```

```

if (nuevaEdad >= 5 && nuevaEdad <= 100) { // Ejemplo de validación
    edad = nuevaEdad;

} else {
    std::cout << "Error: Edad '" << nuevaEdad << "' invalida para el estudiante "
        << nombre << ". Edad no modificada." << std::endl;
}
}

// No ponemos un setter para matricula, asumiendo que no cambia.
// Pero podríamos tener uno si fuera necesario.
void setPromedio(double nuevoPromedio) {
    if (nuevoPromedio >= 0.0 && nuevoPromedio <= 10.0) { // Asumiendo escala 0-10
        promedio = nuevoPromedio;
    } else {
        std::cout << "Error: Promedio '" << nuevoPromedio << "' invalido para "
            << nombre << ". Promedio no modificado." << std::endl;
    }
}

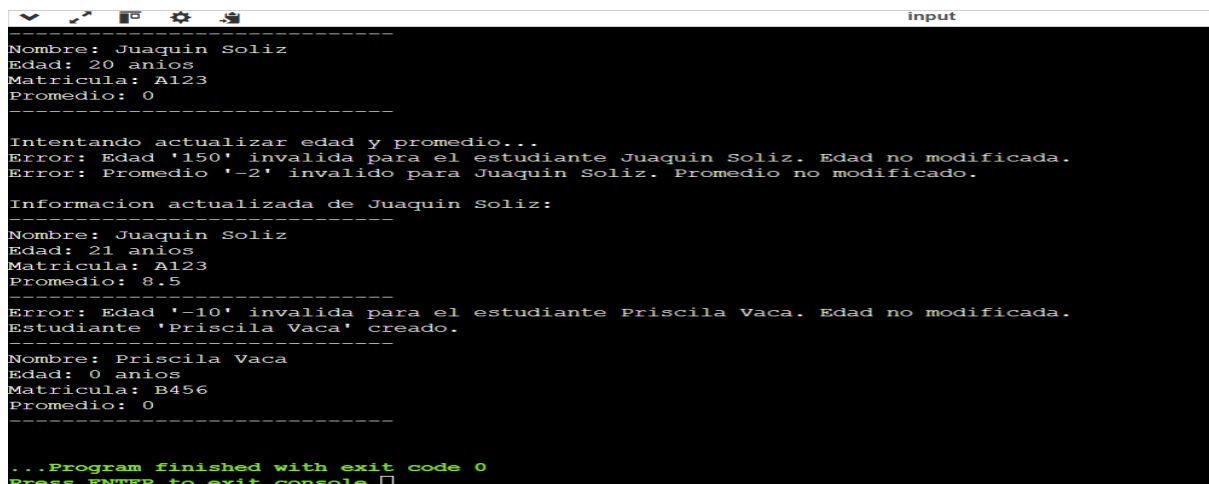
// Otros métodos públicos
void mostrarInformacion() const {
    std::cout << "-----" << std::endl;
    std::cout << "Nombre: " << nombre << std::endl;
    std::cout << "Edad: " << edad << " años" << std::endl;
    std::cout << "Matricula: " << matricula << std::endl;
    std::cout << "Promedio: " << promedio << std::endl;
    std::cout << "-----" << std::endl;
}

}; // Fin de la clase Estudiante

int main() {
    Estudiante estudiante1("Juaquin Soliz", 20, "A123");
    estudiante1.mostrarInformacion();
    // Intentando modificar datos
    // estudiante1.edad = 21; // ¡ERROR! 'edad' es private.
    std::cout << "\nIntentando actualizar edad y promedio..." << std::endl;
    estudiante1.setEdad(21); // Uso correcto del setter
    estudiante1.setPromedio(8.5);
    estudiante1.setEdad(150); // Intentando edad inválida
    estudiante1.setPromedio(-2.0); // Intentando promedio inválido
    std::cout << "\nInformacion actualizada de " << estudiante1.getNombre() << ":" << std::endl;
    estudiante1.mostrarInformacion();
    Estudiante estudiante2("Priscila Vaca", -10, "B456"); // Prueba de validación en constructor
    estudiante2.mostrarInformacion(); // Veremos qué edad tiene Luis
    return 0;
}

```

VER EN PANTALLA



```

input
-----
Nombre: Juaquin Soliz
Edad: 20 años
Matricula: A123
Promedio: 0
-----

Intentando actualizar edad y promedio...
Error: Edad '150' invalida para el estudiante Juaquin Soliz. Edad no modificada.
Error: Promedio '-2' invalido para Juaquin Soliz. Promedio no modificado.

Informacion actualizada de Juaquin Soliz:
-----
Nombre: Juaquin Soliz
Edad: 21 años
Matricula: A123
Promedio: 8.5
-----

Error: Edad '-10' invalida para el estudiante Priscila Vaca. Edad no modificada.
Estudiante 'Priscila Vaca' creado.
-----
Nombre: Priscila Vaca
Edad: 0 años
Matricula: B456
Promedio: 0
-----

...Program finished with exit code 0
Press ENTER to exit console.

```

ENCAPSULANDO ESTUDIANTE II

CODIGOII

```
#include <iostream>
#include <string>

class Estudiante {
private: // DATOS PROTEGIDOS
    std::string nombre;
    int edad;
    std::string matricula;
    double promedio;
    double notaCursada; // Nueva nota de 0-10

public:
    // Constructor
    Estudiante(std::string nom, int ed, std::string matr) {
        nombre = nom;
        // Usamos el setter para edad para aprovechar la validación
        setEdad(ed); // ¡Llamando a nuestro propio setter!
        matricula = matr;
        promedio = 0.0; // Promedio inicial
        notaCursada = 0.0; // Nota inicial
        std::cout << "Estudiante " << nombre << " creado." << std::endl;
    }

    // Getters
    std::string getNombre() const { return nombre; }
    int getEdad() const { return edad; }
    std::string getMatricula() const { return matricula; }
    double getPromedio() const { return promedio; }
    double getNotaCursada() const { return notaCursada; }

    // Setters (con validación donde aplique)
    void setNombre(const std::string& nuevoNombre) {
        if (!nuevoNombre.empty()) {
            nombre = nuevoNombre;
        } else {
            std::cout << "Error: El nombre no puede estar vacio." << std::endl;
        }
    }

    void setEdad(int nuevaEdad) {
        if (nuevaEdad >= 5 && nuevaEdad <= 100) { // Ejemplo de validación
            edad = nuevaEdad;
        } else {
            std::cout << "Error: Edad " << nuevaEdad << " invalida para el estudiante "
                << nombre << ". Edad no modificada." << std::endl;
        }
    }

    void setPromedio(double nuevoPromedio) {
        if (nuevoPromedio >= 0.0 && nuevoPromedio <= 10.0) { // Asumiendo escala 0-10
            promedio = nuevoPromedio;
        } else {
            std::cout << "Error: Promedio " << nuevoPromedio << " invalido para "
                << nombre << ". Promedio no modificado." << std::endl;
        }
    }
}
```

```
void setNotaCursada(double nuevaNota) {
    if (nuevaNota >= 0.0 && nuevaNota <= 10.0) {

        notaCursada = nuevaNota;
        std::cout << "Nota cursada actualizada a: " << notaCursada << std::endl;
    } else {
        std::cout << "Error: Nota cursada " << nuevaNota << " invalida para "
            << nombre << ". Debe estar entre 0 y 10." << std::endl;
    }
}

// Método para determinar si pasó el curso
bool pasoCurso() const {
    return notaCursada >= 5.0;
}

// Método para obtener el estado del curso como string
std::string getEstadoCurso() const {
    if (notaCursada >= 5.0) {
        return "APROBADO - Pasaste de curso";
    } else {
        return "REPROBADO - No cursaste";
    }
}

// Otros métodos públicos
void mostrarInformacion() const {
    std::cout << "-----" << std::endl;
    std::cout << "Nombre: " << nombre << std::endl;
    std::cout << "Edad: " << edad << " años" << std::endl;
    std::cout << "Matricula: " << matricula << std::endl;
    std::cout << "Promedio: " << promedio << std::endl;
    std::cout << "Nota Cursada: " << notaCursada << "/10" << std::endl;
    std::cout << "Estado: " << getEstadoCurso() << std::endl;
    std::cout << "-----" << std::endl;
}
}; // Fin de la clase Estudiante

int main() {
    Estudiante estudiante1("Juaquin Soliz", 20, "A123");
    estudiante1.mostrarInformacion();

    // Intentando modificar datos
    std::cout << "\nIntentando actualizar edad, promedio y nota cursada..." << std::endl;
    estudiante1.setEdad(21); // Uso correcto del setter
    estudiante1.setPromedio(8.5);
    estudiante1.setNotaCursada(7.5); // Nota aprobatoria para aprobar curso
    estudiante1.setEdad(150); // Error Intentando edad inválida
    estudiante1.setPromedio(-2.0); // Error Intentando promedio inválido
    estudiante1.setNotaCursada(15.0); // Error Intentando nota inválida

    std::cout << "\nInformacion actualizada de " << estudiante1.getNombre() << ":" << std::endl;
    estudiante1.mostrarInformacion();

    // Creando Priscila con edad promedio válida
    Estudiante estudiante2("Priscila Vaca", 19, "B456"); // Edad promedio válida
    estudiante2.setNotaCursada(4.2); // Nota mala no aprueba
    estudiante2.setPromedio(6.8);
```



```
std::cout << "\nInformacion de Priscila:" << std::endl;
estudiante2.mostrarInformacion();

// Probando con otra nota para Priscila
std::cout << "\nPriscila mejora su nota:" << std::endl;
estudiante2.setNotaCursada(6.5); // Ahora aprueba
estudiante2.mostrarInformacion();

// Creando un tercer estudiante en el límite
Estudiante estudiante3("Ana Garcia", 22, "C789");
estudiante3.setNotaCursada(5.0); // Justo en el límite para aprobar
estudiante3.setPromedio(7.2);

std::cout << "\nInformacion de Ana (nota limite):" << std::endl;
estudiante3.mostrarInformacion();

return 0;
}
```

VISTO EN CONSOLA

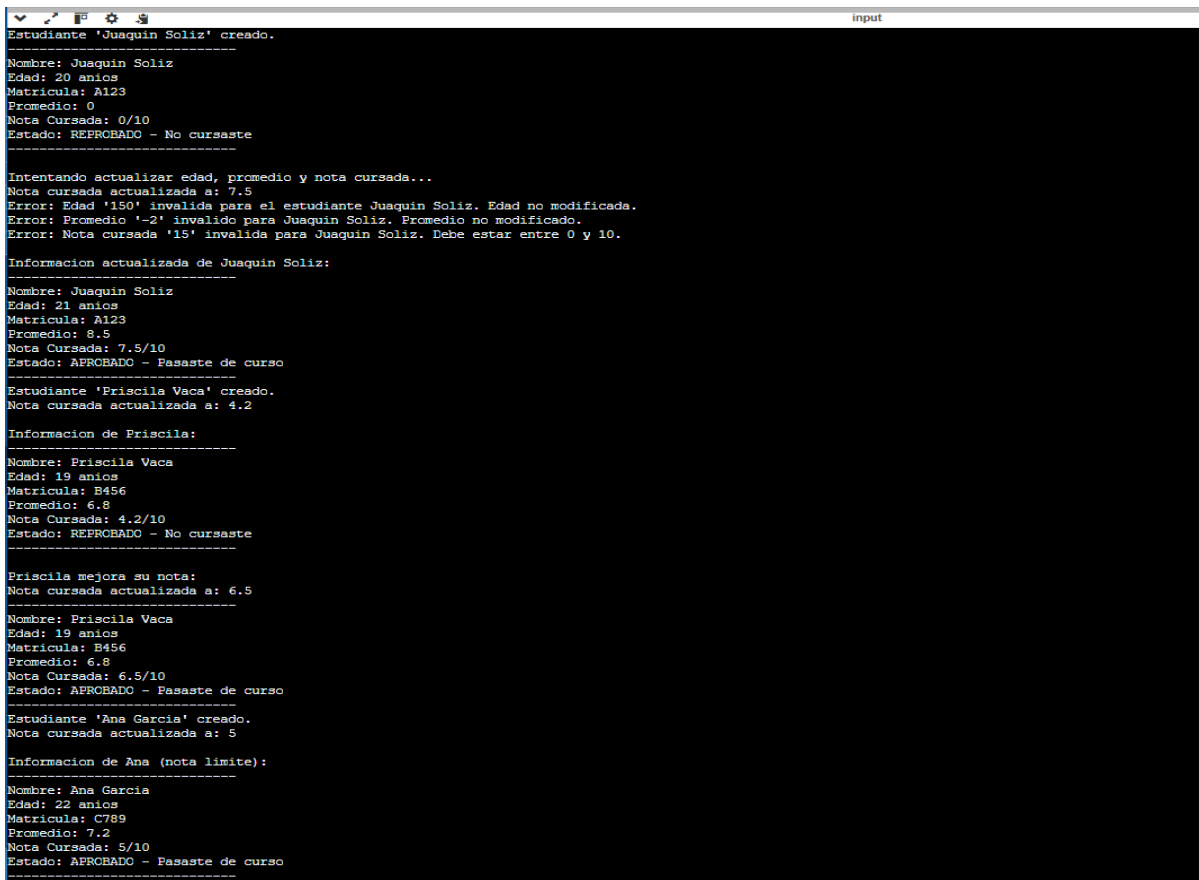
Intentando actualizar edad, promedio y nota cursada...

Nota cursada actualizada a: 7.5

Error: Edad '150' invalida para el estudiante Juaquin Soliz. Edad no modificada.

Error: Promedio '-2' invalido para Juaquin Soliz. Promedio no modificado.

Error: Nota cursada '15' invalida para Juaquin Soliz. Debe estar entre 0 y 10.



```
Estudiante 'Juaquin Soliz' creado.
Nombre: Juaquin Soliz
Edad: 20 años
Matricula: A123
Promedio: 0
Nota Cursada: 0/10
Estado: REPROBADO - No cursaste

Intentando actualizar edad, promedio y nota cursada...
Nota cursada actualizada a: 7.5
Error: Edad '150' invalida para el estudiante Juaquin Soliz. Edad no modificada.
Error: Promedio '-2' invalido para Juaquin Soliz. Promedio no modificado.
Error: Nota cursada '15' invalida para Juaquin Soliz. Debe estar entre 0 y 10.

Informacion actualizada de Juaquin Soliz:
Nombre: Juaquin Soliz
Edad: 21 años
Matricula: A123
Promedio: 8.5
Nota Cursada: 7.5/10
Estado: APROBADO - Pasaste de curso

Estudiante 'Priscila Vaca' creado.
Nota cursada actualizada a: 4.2

Informacion de Priscila:
Nombre: Priscila Vaca
Edad: 19 años
Matricula: B456
Promedio: 6.8
Nota Cursada: 4.2/10
Estado: REPROBADO - No cursaste

Priscila mejora su nota:
Nota cursada actualizada a: 6.5

Nombre: Priscila Vaca
Edad: 19 años
Matricula: B456
Promedio: 6.8
Nota Cursada: 6.5/10
Estado: APROBADO - Pasaste de curso

Estudiante 'Ana Garcia' creado.
Nota cursada actualizada a: 5

Informacion de Ana (nota limite):
Nombre: Ana Garcia
Edad: 22 años
Matricula: C789
Promedio: 7.2
Nota Cursada: 5/10
Estado: APROBADO - Pasaste de curso
```

Constructores: Dando Vida y Estado Inicial a los Objetos

EJEMPLO revisemos en plataforma usoConstructores.cpp

```
class Motor {
public:
    int cilindros;
    Motor(int c) : cilindros(c) { std::cout << "Motor(int) creado" << std::endl; }
    Motor() : cilindros(4) { std::cout << "Motor() por defecto creado" << std::endl; }
};

class Auto {
public:
    std::string marca;
    Motor miMotor; // Objeto miembro
    // Usando lista de inicializadores para miMotor
    Auto(std::string m, int cil) : marca(m), miMotor(cil) {
        std::cout << "Auto creado" << std::endl;
    }
    // Si NO usamos lista para miMotor (menos eficiente):
    Auto(std::string m, int cil) {
        marca = m;
        // miMotor se crea aquí con Motor() por defecto
        // y LUEGO se reasigna o se necesitaría un miMotor.setCi...(cil)
        // Lo cual no es ideal si Motor no tuviera setter o fuera const.
        // Si Motor solo tuviera Motor(int), esto daría error.
        miMotor = Motor(cil); // Esto es reasignación, no inicialización directa.
        std::cout << "Auto creado (forma menos eficiente para motor)" << std::endl;
    }
};
```

compilation failed due to following error(s).

```
in.cpp: In constructor 'Motor::Motor(int)':
in.cpp:4:40: error: 'cout' is not a member of 'std'
  4 |     Motor(int c) : cilindros(c) { std::cout << "Motor(int) creado" << std::endl; }
    |                                     ^~~~~~
in.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
+++ |+#include <iostream>
  1 | class Motor {
in.cpp:4:76: error: 'endl' is not a member of 'std'
  4 |     Motor(int c) : cilindros(c) { std::cout << "Motor(int) creado" << std::endl; }
    |                                     ^~~~~~
in.cpp:1:1: note: 'std::endl' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
+++ |+#include <iostream>
  1 | class Motor {
in.cpp: In constructor 'Motor::Motor()':
in.cpp:5:35: error: 'cout' is not a member of 'std'
  5 |     Motor() : cilindros(4) { std::cout << "Motor() por defecto creado" << std::endl; }
    |                                   ^~~~~~
in.cpp:5:35: note: 'std::cout' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
in.cpp:5:80: error: 'endl' is not a member of 'std'
  5 |     Motor() : cilindros(4) { std::cout << "Motor() por defecto creado" << std::endl; }
    |                                     ^~~~~~
in.cpp:5:80: note: 'std::endl' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
in.cpp: At global scope:
in.cpp:9:10: error: 'string' in namespace 'std' does not name a type
  9 |     std::string marca;
```

COMPLETANDO EL EJEMPLO DANDO VIDA AL CONTRUCTOR

CODIGO II

```
#include <iostream>
#include <string>

class Motor {
public:
    int cilindros;
    std::string tipoCombustible;
    float potencia;

    Motor(int c, std::string t = "gasolina", float p = 100.0f)
        : cilindros(c), tipoCombustible(t), potencia(p) {
        std::cout << "Motor creado con " << cilindros << " cilindros" << std::endl;
    }

    Motor() : cilindros(4), tipoCombustible("gasolina"), potencia(120.0f) {
        std::cout << "Motor por defecto creado" << std::endl;
    }

    void mostrarEspecificaciones() {
        std::cout << "Motor: " << cilindros << " cilindros, "
            << tipoCombustible << ", " << potencia << " HP" << std::endl;
    }
};

class Auto {
public:
    std::string marca;
    std::string modelo;
    int año;
    Motor miMotor;

    // Constructor principal con parámetros por defecto
    Auto(std::string m, std::string mod, int a, int cil, std::string comb, float pot)
        : marca(m), modelo(mod), año(a), miMotor(cil, comb, pot) {
        std::cout << "Auto creado (inicialización eficiente)" << std::endl;
    }

    // Constructor delegado
    Auto(std::string m, std::string mod, int a, int cil)
        : Auto(m, mod, a, cil, "gasolina", 100.0f) {
        std::cout << "Auto creado usando valores por defecto" << std::endl;
    }

    void mostrarInfo() {
        std::cout << "\n" << marca << " " << modelo << " (" << año << ")" << "\n";
        miMotor.mostrarEspecificaciones();
    }
};

int main() {

    Auto auto1("Toyota", "Corolla", 2022, 4, "hibrido", 150.0f);
    auto1.mostrarInfo();

    Auto auto2("Ford", "Mustang", 2023, 8); // Usa el constructor delegado
    auto2.mostrarInfo();
    std::cout << " ACTUALIZADO POR DOUGLAS FLOR HERNANDEZ" << std::endl;
    return 0;
}
```

VISTO EN CONSOLA

```
Motor creado con 4 cilindros
Auto creado (inicialización eficiente)

Toyota Corolla (2022)
Motor: 4 cilindros, hibrido, 150 HP
Motor creado con 8 cilindros
Auto creado (inicialización eficiente)
Auto creado usando valores por defecto

Ford Mustang (2023)
Motor: 8 cilindros, gasolina, 100 HP
ACTUALIZADO POR DOUGLAS FLOR HERNANDEZ

...Program finished with exit code 0
Press ENTER to exit console.[]
```

La clase motor mejorada : como atributos añadidos potencia y tipoCombustible constructores igual mejorados para manejar los nuevos atributos como ser mostrarEspecificaciones() y aumentarPotencia()

la clase auto : como atributos modelo y año (anio) el constructor principal mejorado para una inicializacion mejorada para prevenir ambigüedad con los constructores

en el Main : ya muestra la creación de los autos de las dos formas nos demuestra la interaccion de cada clase entre motor y auto

Ejemplo en C++ para ilustrar el uso de constructores en una clase

```
main.cpp
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Clase Punto para representar coordenadas en 2D
6 class Punto {
7 private:
8     double x;
9     double y;
10
11 public:
12     // Constructor por defecto (sin argumentos)
13     Punto() : x(0.0), y(0.0) {
14         cout << "Constructor por defecto llamado. x = " << x << ", y = " << y << endl;
15     }
16
17     // Constructor con parámetros
18     Punto(double xVal, double yVal) : x(xVal), y(yVal) {
19         cout << "Constructor con parámetros llamado. x = " << x << ", y = " << y << endl;
20     }
21
22     // Constructor copia (recibe otro Punto por referencia constante)
23     Punto(const Punto& otro) : x(otro.x), y(otro.y) {
24         cout << "Constructor copia llamado. Copiando punto: x = " << x << ", y = " << y << endl;
25     }
26
27     // Método para mostrar el punto
28     void mostrar() {
29         cout << "Punto en coordenadas: (" << x << ", " << y << ")" << endl;
30     }
31 };
32
33 // Función principal
34 int main() {
35     // Crear un punto con el constructor por defecto
36     Punto p1;
37     p1.mostrar();
38
39     // Crear un punto con valores personalizados
40     Punto p2(5.5, 7.3);
41     p2.mostrar();
42
43     // Crear un punto copiando otro (constructor copia)
44     Punto p3(p2);
45     p3.mostrar();
46
47     return 0;
48 }
input
Constructor por defecto llamado. x = 0, y = 0
Punto en coordenadas: (0, 0)
Constructor con parámetros llamado. x = 5.5, y = 7.3
Punto en coordenadas: (5.5, 7.3)
Constructor copia llamado. Copiando punto: x = 5.5, y = 7.3
Punto en coordenadas: (5.5, 7.3)
...Program finished with exit code 0
Press ENTER to exit console.
```

Actividad (Diseño Rápido en Papel/Grupos)

1. **Escenario:** Pensemos en el ejemplo simple de una clase, por ejemplo:
 - o **Clase Videojuego:** Necesita almacenar **titulo** (string), **genero** (string), **calificacionESRB** (string, ej. "E", "T", "M"), **anioLanzamiento** (int), **puntuacionMedia** (double, 0-10).
2. **Tarea para los Grupos:**
 - o Decidan qué atributos deben ser **private**. (¡Probablemente todos!).
 - o Diseñen los getters públicos necesarios. ¿Para todos?
 - o Diseñen los setters públicos necesarios. ¿Para todos? ¿Cuáles necesitarían validación y qué tipo de validación?
 - o (Opcional) ¿Algún método **private** de ayuda que se les ocurra?
3. **Compartan al frente:** Un par de grupos comparten su diseño de interfaz pública (getters/setters) y justifican sus decisiones de validación.

La idea es practicar la toma de decisiones sobre qué encapsular y cómo exponerlo.

respuesta de la actividad grupal

Punteros Letales

video juegos	
Atributos - titulo : string - genero : string - calificacionESRB : string -anioLanzamiento: int -puntuacionMedia: double	Encapsulamiento (atributos privados): Todos los atributos son privados para proteger la integridad de los datos Esto permite controlar cómo se modifican los valores
Métodos <pre> +getTitulo(): string + setTitulo(string): + getGenero(): string + setGenero(string): + getCalificacionESRB(): string + setCalificacionESRB(string): + getAnioLanzamiento(): int + setAnioLanzamiento(int): + getPuntuacionMedia(): + setPuntuacionMedia(double): - validarESRB(string): - validarPuntuacion(double): </pre>	Getters públicos: Proporcionados para todos los atributos ya que es información que puede ser consultada No modifican el estado interno del objeto Setters con validación: setTitulo y setGenero son simples (sin validación compleja) setCalificacionESRB, setAnioLanzamiento y setPuntuacionMedia incluyen validación: ESRB debe ser uno de los valores permitidos ("E", "E10+", "T", "M", "A", "RP") Año de lanzamiento debe ser razonable (ej. 1950-2025) Puntuación debe estar entre 0 y 10 Métodos privados de ayuda: validarESRB y validarPuntuacion para reutilizar la lógica de validación Justificación de las Decisiones Encapsulamiento: Protege los datos internos de modificaciones no controladas Validaciones en setters: Garantiza la integridad de los datos en todo momento Métodos privados de ayuda: Mejora la legibilidad y reutilización del código Setters que retornan bool: Permite al código cliente saber si la operación fue exitosa Getters constantes: No modifican el objeto y pueden usarse en contextos const

pasando a código la clases con los métodos

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

class Videojuego {
private:
    std::string titulo;
    std::string genero;
    std::string calificacionESRB;
    int anioLanzamiento;
    double puntuacionMedia;

    bool validarESRB(const std::string& esrb) {
        const std::vector<std::string> valoresValidos = {"E", "E10+", "T", "M", "A", "RP"};
        return std::find(valoresValidos.begin(), valoresValidos.end(), esrb) != valoresValidos.end();
    }

    bool validarPuntuacion(double puntuacion) {
        return puntuacion >= 0.0 && puntuacion <= 10.0;
    }

public:
    // Getters
    std::string getTitulo() const { return titulo; }
    std::string getGenero() const { return genero; }
    std::string getCalificacionESRB() const { return calificacionESRB; }
    int getAnioLanzamiento() const { return anioLanzamiento; }
    double getPuntuacionMedia() const { return puntuacionMedia; }

    // Setters
    void setTitulo(const std::string& t) { titulo = t; }
    void setGenero(const std::string& g) { genero = g; }

    bool setCalificacionESRB(const std::string& esrb) {
        if (validarESRB(esrb)) {
            calificacionESRB = esrb;
            return true;
        }
        return false;
    }

    bool setAnioLanzamiento(int anio) {
        if (anio >= 1950 && anio <= 2025) {
            anioLanzamiento = anio;
            return true;
        }
        return false;
    }

    bool setPuntuacionMedia(double puntuacion) {
        if (validarPuntuacion(puntuacion)) {
            puntuacionMedia = puntuacion;
            return true;
        }
        return false;
    }

    // Método para mostrar información
```

```
void mostrarInfo() const {
    std::cout << "Título: " << titulo << "\n"
        << "Género: " << genero << "\n"
        << "Clasificación ESRB: " << calificacionESRB << "\n"
        << "Año de lanzamiento: " << anioLanzamiento << "\n"
        << "Puntuación media: " << puntuacionMedia << "/10\n";
}

int main() {
    Videojuego juego;

    // Setters con validación
    juego.setTitulo("The Legend of Zelda: Breath of the Wild");
    juego.setGenero("Aventura");

    if (!juego.setCalificacionESRB("E")) {
        std::cout << "Clasificación ESRB no válida\n";
    }

    if (!juego.setAnioLanzamiento(2017)) {
        std::cout << "Año de lanzamiento no válido\n";
    }

    if (!juego.setPuntuacionMedia(9.7)) {
        std::cout << "Puntuación no válida (debe ser 0-10)\n";
    }

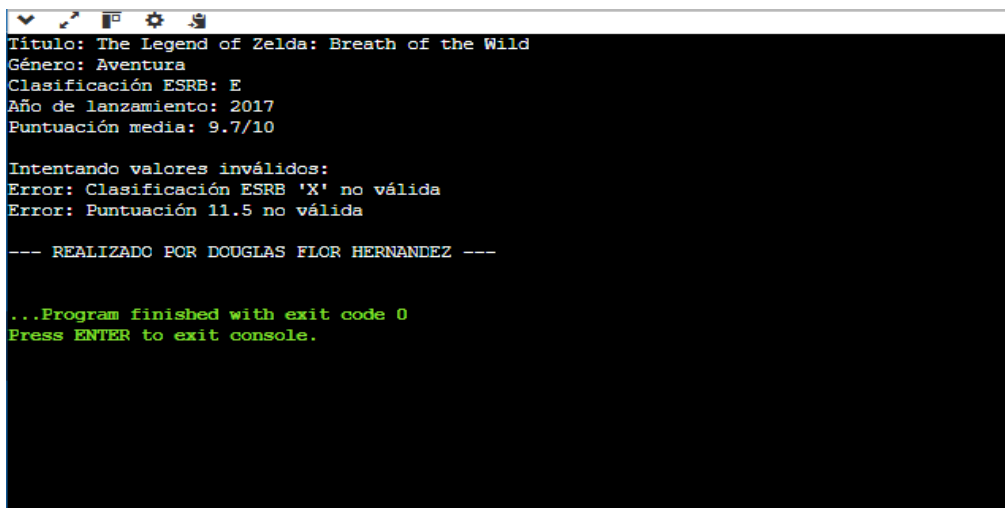
    // Mostrar información
    juego.mostrarInfo();

    // Intentar valores inválidos
    std::cout << "\nIntentando valores inválidos:\n";
    if (!juego.setCalificacionESRB("X")) {
        std::cout << "Error: Clasificación ESRB 'X' no válida\n";
    }

    if (!juego.setPuntuacionMedia(11.5)) {
        std::cout << "Error: Puntuación 11.5 no válida\n";
    }

    return 0;
}
```

vista en consola



```
Título: The Legend of Zelda: Breath of the Wild
Género: Aventura
Clasificación ESRB: E
Año de lanzamiento: 2017
Puntuación media: 9.7/10

Intentando valores inválidos:
Error: Clasificación ESRB 'X' no válida
Error: Puntuación 11.5 no válida

--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---

...Program finished with exit code 0
Press ENTER to exit console.
```

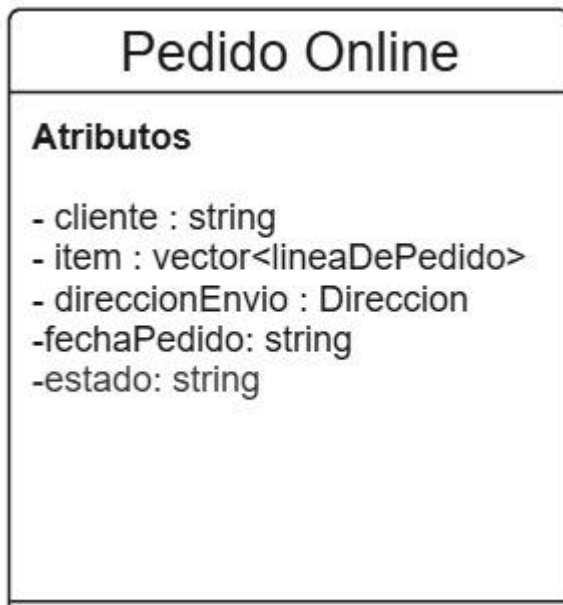

Fecha y hora actual: 2025-06-20 00:01:45

```
#include <iostream>
#include <string>
// Clase 'Parte': Motor
class Motor {
private:
    int caballosDeFuerza;
    bool encendido;
public:
    Motor(int hp = 150) : caballosDeFuerza(hp), encendido(false) {
        std::cout << " CONSTRUCTOR Motor: Creado motor de " << caballosDeFuerza << " HP." << std::endl;
    }
    ~Motor() {
        std::cout << " DESTRUCTOR Motor: Destruido motor de " << caballosDeFuerza << " HP." << std::endl;
    }
    void arrancar() {
        if (!encendido) { encendido = true; std::cout << " Motor: ¡BRUM! Encendido." << std::endl; }
        else { std::cout << " Motor: Ya estaba encendido." << std::endl; }
    }
    void detener() {
        if (encendido) { encendido = false; std::cout << " Motor: ...silencio. Apagado." << std::endl; }
        else { std::cout << " Motor: Ya estaba apagado." << std::endl; }
    }
    void mostrarEstado() const {
        std::cout << " Estado del Motor: " << (encendido ? "Encendido" : "Apagado")
        << ", HP: " << caballosDeFuerza << std::endl;
    }
};
// Clase 'Todo' o 'Contenedora': Automovil
class Automovil {
private:
    std::string marca;
    std::string modelo;
    Motor motorInterno; // ¡COMPOSICIÓN! Un objeto Motor es MIEMBRO de Automovil.
public:
    // Constructor de Automovil: INICIALIZA motorInterno usando la lista de inicializadores
    Automovil(std::string ma, std::string mo, int hpDelMotor)
        : marca(ma), modelo(mo), motorInterno(hpDelMotor) {
        std::cout << "CONSTRUCTOR Automovil: Ensamblado un " << marca << " " << modelo
        << " con un motor." << std::endl;
    }
    ~Automovil() {
        std::cout << "DESTRUCTOR Automovil: Desguazando el " << marca << " " << modelo << "." << std::endl;
        // El destructor de motorInterno se llamará AUTOMÁTICAMENTE aquí.
    }
    void encender() {
        std::cout << modelo << ": Intentando encender..." << std::endl;
        motorInterno.arrancar(); // Delega la acción al objeto Motor
    }
    void apagar() {
        std::cout << modelo << ": Intentando apagar..." << std::endl;
        motorInterno.detener();
    }
    void verDiagnostico() const {
        std::cout << "Diagnostico del " << modelo << ":" << std::endl;
        motorInterno.mostrarEstado();
    }
};
```

```
PROBLEMAS 195 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS DE LA CONSULTA

DESTRUCTOR Motor: Destruído motor de 200 HP.
PS C:\Users\59168\Downloads\output> cd 'c:\Users\59168\Downloads\output'
PS C:\Users\59168\Downloads\output> & .\automovil.exe
--- Creando un Automovil en el Stack ---
CONSTRUCTOR Motor: Creado motor de 200 HP.
CONSTRUCTOR Automovil: Ensamblado un SuperMarca ModeloX con un motor.
Diagnostico del ModeloX:
Estado del Motor: Apagado, HP: 200
ModeloX: Intentando encender...
Motor: ¡IBRUM! Encendido.
Diagnostico del ModeloX:
Estado del Motor: Encendido, HP: 200
ModeloX: Intentando apagar...
Motor: ...silencio. Apagado.

--- Saliendo de main (miAuto se destruira) ---
DESTRUCTOR Automovil: Desguazando el SuperMarca ModeloX.
DESTRUCTOR Motor: Destruído motor de 200 HP.
PS C:\Users\59168\Downloads\output> |
```



CONSTRUCTOR Y DESTRUCTOR DE LA CLASE ANIMAL

```
1- /*****
2
3 Welcome to GDB Online.
4 GDB online is an online compiler and debugger tool for C, C++, Python, PHP, Ruby,
5 C#, OCaml, VB, Perl, Swift, Prolog, Javascript, Pascal, COBOL, HTML, CSS, JS
6 Code, Compile, Run and Debug online from anywhere in world.
7
8 *****/
9 #include <iostream>
10 #include <string>
11
12 // Clase Base
13 class Animal {
14 protected:
15     std::string nombre;
16     int edad;
17
18 public:
19     Animal(const std::string& n, int e) : nombre(n), edad(e) {
20         std::cout << "CONSTRUCTOR Animal: Nace un animal llamado '"
21             << nombre << "' de '" << edad << "' anio(s)." << std::endl;
22     }
23
24     ~Animal() {
25         std::cout << "DESTRUCTOR Animal: Muere el animal '" << nombre << "'." << std::endl;
26     }
27
28     void comer() const {
29         std::cout << nombre << " esta comiendo." << std::endl;
30     }
31
32     void dormir() const {
33         std::cout << nombre << " esta durmiendo." << std::endl;
34     }
35
36     std::string getNombre() const { return nombre; }
37 };
38
39 // Clase Derivada
40 class Perro : public Animal {
41 private:
42     std::string raza;
43
44 public:
45     Perro(const std::string& n, int e, const std::string& r)
46         : Animal(n, e), raza(r) {
47         std::cout << "CONSTRUCTOR Perro: Nace un perro de raza '"
48             << raza << "', que tambien es un animal llamado '"
49             << nombre << "'." << std::endl;
50     }
51
52     ~Perro() {
53         std::cout << "DESTRUCTOR Perro: Muere el perro '" << nombre
54             << "' de raza '" << raza << "'." << std::endl;
55     }
56
57     void ladrar() const {https://www.onlinegdb.com/#tab-stdin
58         std::cout << getNombre() << " (un " << raza << ") dice: ¡Guau! ¡Guau!" << std::endl;
```

88

RESPUESTA EN CONSOLA

```
--- Creando un Animal generico ---
CONSTRUCTOR Animal: Nace un animal llamado 'Alex el Leon' de 10 anio(s).
Alex el Leon esta comiendo.

--- Creando un Perro ---
CONSTRUCTOR Animal: Nace un animal llamado 'Buddy' de 3 anio(s).
CONSTRUCTOR Perro: Nace un perro de raza 'Golden Retriever', que tambien es un animal llamado 'Buddy'.
Buddy esta comiendo.
Buddy esta durmiendo.
El nombre de mi mascota es: Buddy
Buddy (un Golden Retriever) dice: ¡Guau! ¡Guau!

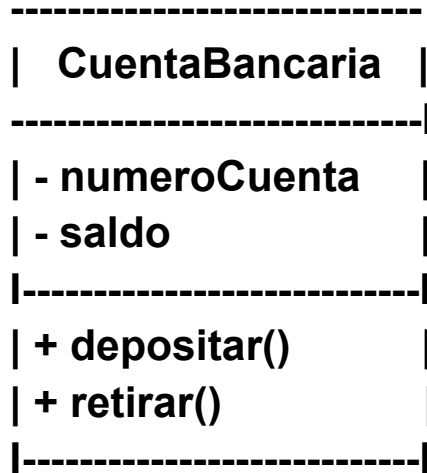
--- Fin de main (los objetos se destruiaran) ---

--- REALIZADO POR DOUGLAS FLOR HERNANDEZ ---
DESTRUCTOR Perro: Muere el perro 'Buddy' de raza 'Golden Retriever'.
DESTRUCTOR Animal: Muere el animal 'Buddy'.
DESTRUCTOR Animal: Muere el animal 'Alex el Leon'.

...Program finished with exit code 0
Press ENTER to exit console.
```

88

Descripción del Diagrama de Clases



CuentaBancaria es la clase base con atributos comunes numeroCuenta y saldo, y métodos comunes depositar() y retirar().

CuentaAhorros tiene un atributo específico tasaInteres y un método calcularInteres().

CuentaCorriente tiene un atributo específico limiteSobregiro y un método usarSobregiro().

CuentaPlazoFijo tiene un atributo específico plazo y un método calcularInteres() (diferente de **CuentaAhorros**, pero con mismo nombre para mostrar polimorfismo).

FIGURAS Y CIRCULOS EJECUTANDO EN C++

```
main.cpp
1  #include <iostream>
2  #include <string>
3  #define PI 3.14159
4
5  struct Figura {
6  protected:
7      std::string color;
8      std::string nombreFigura;
9  public:
10     Figura(std::string c, std::string nf) : color(c), nombreFigura(nf) {
11         std::cout << " CONSTRUCTOR Figura: " << nombreFigura << " de color " << color << std::endl;
12     }
13     // Ahora es virtual para permitir override real
14     virtual void dibujar() const {
15         std::cout << "Figura " << nombreFigura << ": Dibujando una figura geometrica generica de color " << color << "." << std::endl;
16     }
17     virtual ~Figura() {
18         std::cout << " DESTRUCTOR Figura: " << nombreFigura << " " << std::endl;
19     }
20
21     SOBRESCRITURA del método dibujar()
22     Mismo firma que Figura::dibujar()
23     Añadimos 'override' (C++11) para seguridad del compilador
24     struct Circulo : public Figura {
25     public:
26         double radio;
27     public:
28         Circulo(std::string c, double r) : Figura(c, "Circulo"), radio(r) {
29             std::cout << " CONSTRUCTOR Circulo: Radio " << radio << std::endl;
30         }
31         void dibujar() const override {
32             std::cout << "Circulo " << nombreFigura << ": Dibujando un circulo perfecto de color " << color << " y radio " << radio << "." << std::endl;
33             std::cout << " Area: " << (PI * radio * radio) << std::endl;
34         }
35         ~Circulo() override {
36             std::cout << " DESTRUCTOR Circulo: Radio " << radio << std::endl;
37         }
38     };
39
40     struct Rectangulo : public Figura {
41     public:
42         double base, altura;
43     public:
44         Rectangulo(std::string c, double b, double h) : Figura(c, "Rectangulo"), base(b), altura(h) {
45             std::cout << " CONSTRUCTOR Rectangulo: Base " << b << ", Altura " << h << std::endl;
46         }
47         void dibujar() const override {
48             std::cout << "Rectangulo " << nombreFigura << ": Dibujando un rectangulo de color " << color << " con base " << base << " y altura " << altura << "." << std::endl;
49             std::cout << " Area: " << (base * altura) << std::endl;
50         }
51         ~Rectangulo() override {
52             std::cout << " DESTRUCTOR Rectangulo: Base " << base << ", altura " << altura << std::endl;
53         }
54     };
55
56     main() {
57
58 }
```

VISTO EN CONSOLA

```
--- Creando y dibujando una Figura generica ---
CONSTRUCTOR Figura: 'Figura Misteriosa' de color Azul
Figura 'Figura Misteriosa': Dibujando una figura geometrica generica de color Azul.

--- Creando y dibujando un Circulo ---
CONSTRUCTOR Figura: 'Circulo' de color Rojo
CONSTRUCTOR Circulo: Radio 5
Circulo 'Circulo': Dibujando un circulo perfecto de color Rojo y radio 5.
Area: 78.5397

--- Creando y dibujando un Rectangulo ---
CONSTRUCTOR Figura: 'Rectangulo' de color Verde
CONSTRUCTOR Rectangulo: Base 4, Altura 6
Rectangulo 'Rectangulo': Dibujando un rectangulo de color Verde con base 4 y altura 6.
Area: 24

--- Fin de main ---
DESTRUCTOR Rectangulo: Base 4, Altura 6
DESTRUCTOR Figura: 'Rectangulo'
DESTRUCTOR Circulo: Radio 5
DESTRUCTOR Figura: 'Circulo'
DESTRUCTOR Figura: 'Figura Misteriosa'

...Program finished with exit code 0
Press ENTER to exit console.
```